

MAIE 6000C: Engineering AI Products: From Prototype to Production

Week 2 — Requirements and System Decomposition

Program: MSc in AI & Entrepreneurship

Term: Fall 2026/27

Instructor: Lefteris NTAFLLOS, PhD

The Hong Kong University of Science and Technology



Today's focus

- Why scope quality matters
- How to frame an operational problem
- How to identify the primary user and workflow
- Functional vs non-functional requirements
- System decomposition
- How to choose a Week 7 vertical slice

Prepare for Week 7 Midterm Milestone Demo

Which can your team *actually demo as a vertical slice* by Week 7?

- “AI platform for operations”
- “Ticket triage suggestion workflow for support agents”
- Why?

Week 7 is a **Midterm Milestone Demo**. You must show a *live* end-to-end path:

- a request/input enters the system
- **one persisted state change**
- **one async/worker step**
- **one persisted output/result**
- **one operational signal** (logs, status, health, metrics)

Takeaway: Specificity isn’t “less ambitious.” It’s what makes your project demo-able by Week 7.

Requirements begin with the workflow

Before choosing tools, define:

- Who is the user?
- What are they trying to accomplish?
- What enters the system?
- What state must be remembered?
- What decision or output is produced?
- What happens when the system is uncertain or wrong?



Problem framing

A strong problem statement includes:

- Operational context
- Primary user
- Pain point or inefficiency
- Why the workflow matters
- What success looks like
- What is out of scope
- **Example structure:**
For **[user]**, the system helps **[task]** by **[workflow]**, so that **[operational value]**.
- For **[support agents]**, the system helps **[triage incoming tickets]** by **[storing submitted tickets, running background classification, and presenting routing suggestions for review]**, so that **[urgent or misrouted tickets are easier to identify]**.

Weak vs strong problem statements

Weak

AI support platform

Smart facilities dispatch

Predict energy problems

Chatbot for everything

Strong

Triage incoming support tickets for routing review **for support agents;**
reduce misroutes by X%

Classify facilities requests **for facilities dispatch; cut time-to-assign**
from A to B

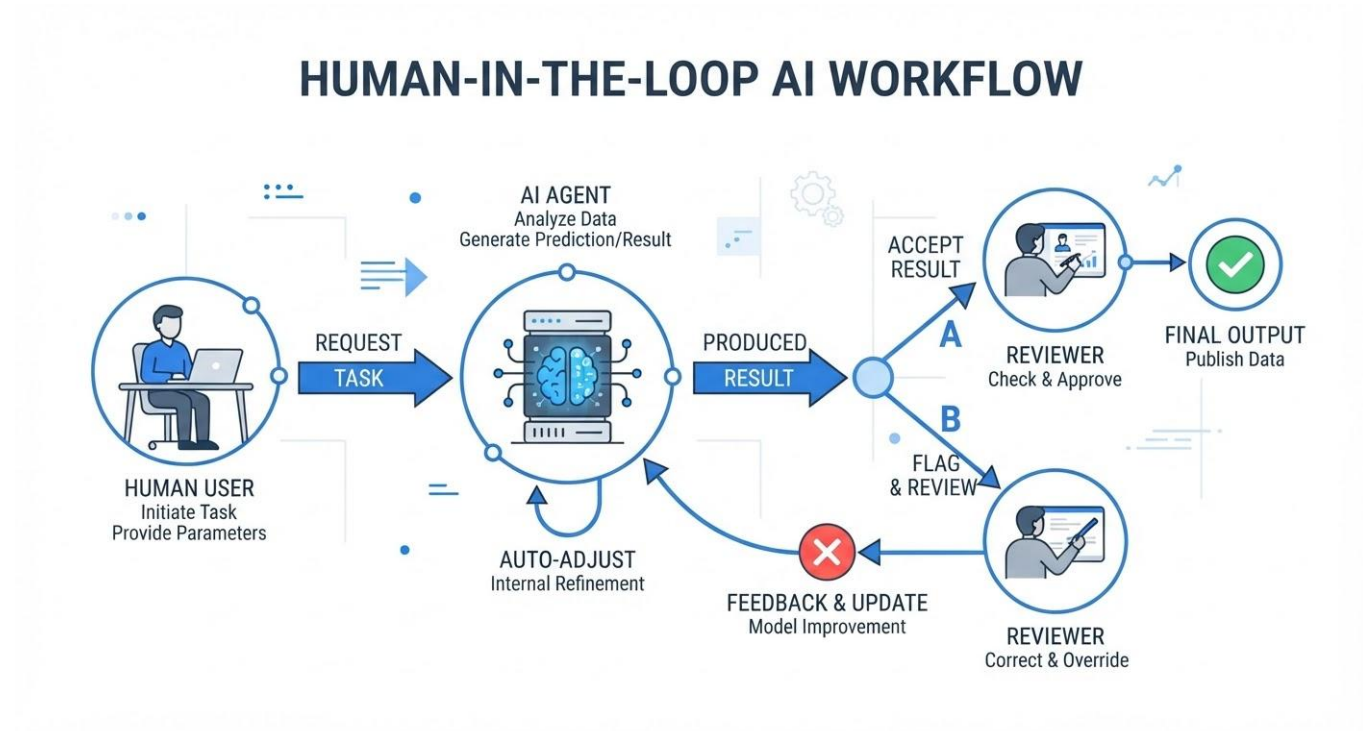
Detect anomalous energy observations and create alerts **for facilities;**
reduce uninvestigated spikes within 24h

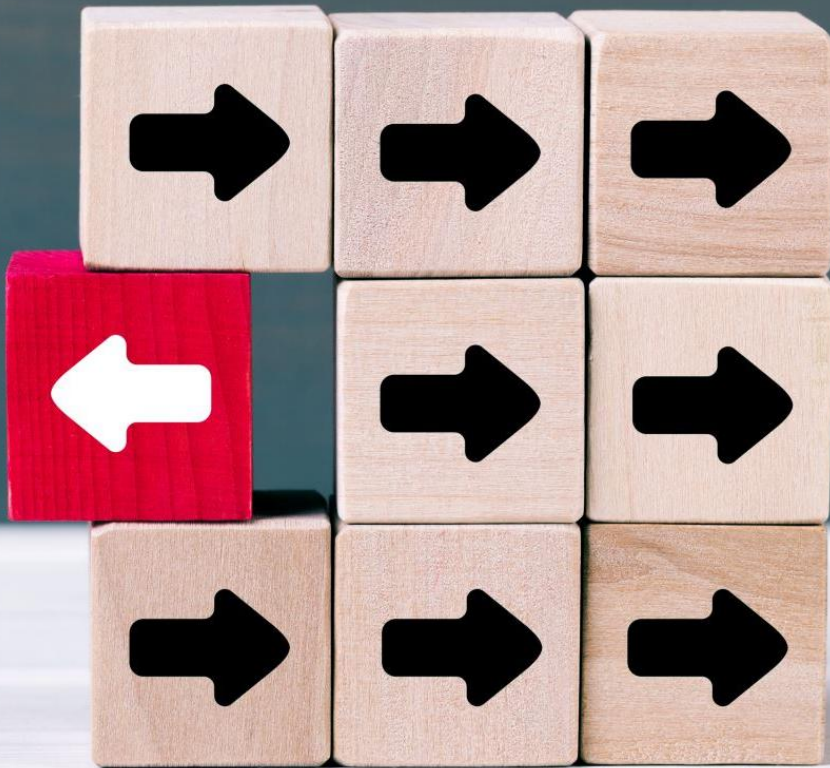
Answer **one** bounded policy question with escalation **for students;**
target $\geq 90\%$ grounded answers; escalate low-confidence

Primary users and workflows

Clarify the workflow by asking:

- Who is the primary user?
- What triggers the workflow?
- What does the user do first?
- What should the system do next?
- What happens if the AI output is wrong?
- Who reviews, corrects, or overrides?





Workflow-first requirements

Start from one operational path

Input (A ticket text, a form submission, an email)



Persistence (A database, a ticketing system, a spreadsheet, a queue)



Background or worker step (deduplication, enrichment, extracting entities)



AI-enabled step (Classify, summarize, draft)



Review (human approval step) **or fallback** (route to manual queue, ask one clarifying question, show policy link + escalate)



Output / status (Ticket assigned, alert created, response sent)



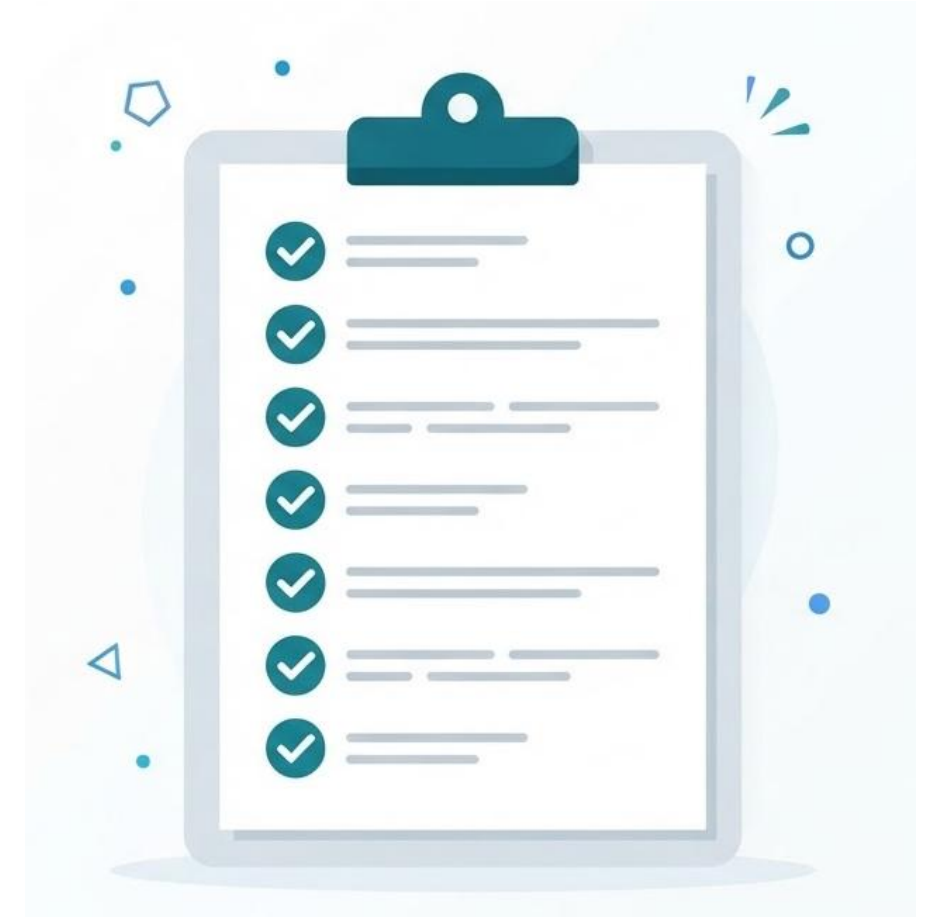
Operational signals (logs/metrics) (dashboards, error rates, queue depth, latency, override rate, misroute rate, investigation SLA, cost per item)

Functional requirements

Functional requirements describe what the system does.

Examples:

- Create a ticket
- Store document metadata
- Create a pending job
- Retrieve candidate evidence
- Show a review queue
- Allow approval or override
- Persist the final decision



- **Functional requirements must contain verbs**
- **Don't confuse the AI step with the system**

Non-functional requirements

- 'Latency under 3 seconds for 95% of student policy queries.'
- 'All overrides are logged with reviewer ID and timestamp.'
- 'Only facilities dispatch can view location-level request details.'
- 'System processes 1,000 tickets/day with <1% job failures.'



Non-functional requirements describe system qualities

(how well the system does its job).

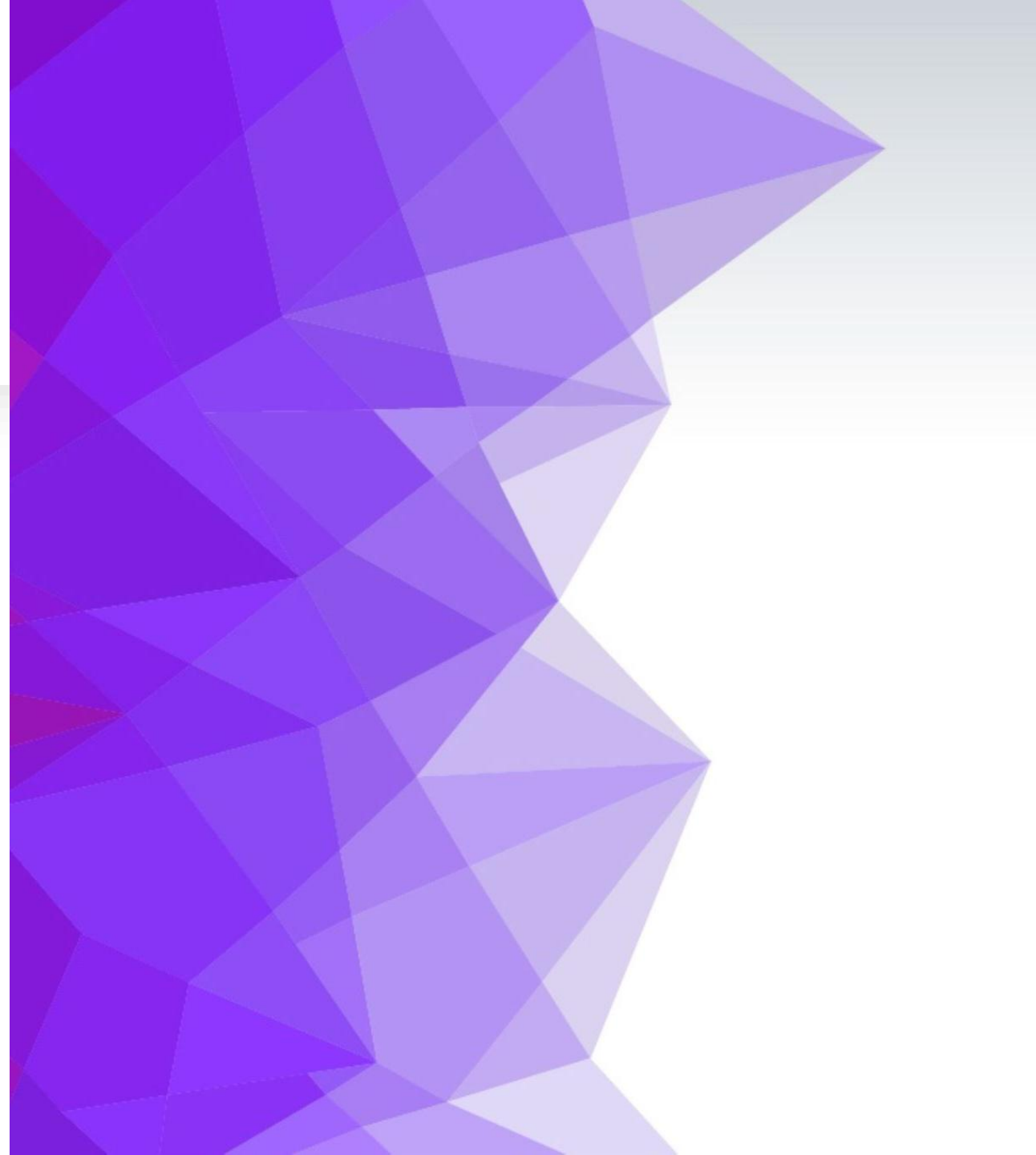
Examples:

- Reproducibility
- Reliability
- Maintainability
- Auditability
- Privacy
- Security
- Latency
- Throughput
- Cost constraints
- Operability

Quick check: functional or non-functional?

Which type is each?

- The system stores uploaded metadata
- Startup works from a clean clone
- A reviewer can override the AI label
- Logs avoid raw sensitive text where possible
- Typical jobs complete within 30 seconds



Persistent entities and state changes

Ask two questions

1. What records must exist?
2. How do they change over time?

Common entities:

- Submitted item
- Job
- AI result
- Review decision
- Alert / status record

Common states:

submitted → queued → processing → completed / reviewed / error

- 
1. What exists immediately after submission?
 2. What exists after background processing?
 3. What exists after human review?
 4. What states should be visible if something fails?

System components

Typical components

- API service
- Database
- Job queue / task scheduler
- Worker
- AI service
- Review interface or API endpoint
- Documentation
- Tests
- Logs / health / metrics

Key question:

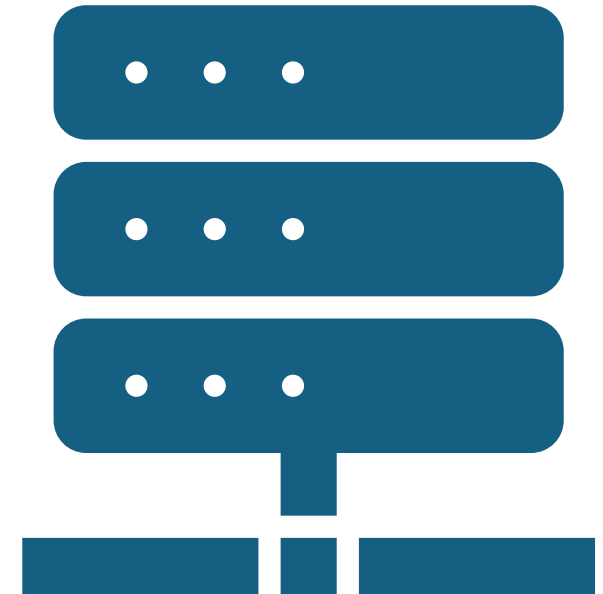
What responsibility belongs where?



Worked example: from brief to system plan

Example: Support ticket triage

- **Input:** ticket title + description (**FastAPI** `api`)
- **Persistence:** `tickets` + `jobs` tables (**PostgreSQL** via **SQLAlchemy**)
- **Queue / scheduler:** `jobs` table acts as the queue (DB-backed job queue)
- **Worker:** background processor service (`worker`)
- **AI step:** internal AI triage service (`ai`)
- **Review:** optional reviewer endpoint/UI (`api`) stores override
- **Output:** routing decision + status persisted (**PostgreSQL**)
- **Signals:** `/health/live` + job status + logs/metrics (**Compose** logs)



Scope control and first scope cuts

A good semester project is:

1. Narrow enough to finish
2. Real enough to demonstrate
3. Integrated early
4. Honest about limitations

First scope cuts often include:

- Fewer users
- One workflow
- Minimal UI
- Simpler model
- Manual fallback
- No paid dependencies



Week 7 vertical slice target

By Week 7, a team should show:

- Request or input entering the system
- One persisted state change
- One async or worker step
- One persisted output or result
- One operational signal
(logs, status, health output)



Today's lab task



In today's lab

- One candidate project direction
- One draft problem statement
- One primary user and trigger event
- One core workflow
- Initial requirements and decomposition notes
- A realistic Week 7 vertical-slice idea
- Initial risks and scope cuts